Output concepts

df.head() : the first five records of the dataset are displayed.

```
import pandas as pd
import numpy as np

# generate patient dataset

np.random.seed(42)
df = pd.DataFrame({
    "patient_id":[f"P{i}" for i in range(1,101)],
    "age":np.random.randint(20,80,100),
    "gender":np.random.choice(["Male","Female"],100),
    "region":np.random.choice(["North","South","East","West"],100),
    "bmi":np.random.normal(25,5,100),
    "blood_pressure":np.random.normal(120,15,100),
    "cholesterol":np.random.normal(200,30,100),
    "glucose":np.random.normal(100,20,100),
    "disease_risk":np.random.choice([0,1],100)
})
df.to_csv("patient_health_records.csv",index=False)
```

```
df.head()
```

	patient_id	age	gender	region	bmi	blood_pressure	cholesterol	glucose	disease_risk
0	P1	58	Female	East	23.928378	111.660827	159.730615	99.967281	0
1	P2	71	Male	East	21.554029	118.083978	99.077134	91.479353	0
2	P3	48	Female	North	28.639166	96.591644	233.706253	125.509265	1
3	P4	34	Male	West	21.904653	150.005221	264.296156	98.168679	0
4	P5	62	Female	North	26.780799	101.998024	189.722334	118.741450	0

Data exploration: - dataset information

```python
# initial data exploration

# dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   patient_id      100 non-null    object
 1   age             94 non-null     float64
 2   gender          94 non-null     object
 3   region          94 non-null     object
 4   bmi             94 non-null     float64
 5   blood_pressure  100 non-null    float64
 6   cholesterol     94 non-null     float64
 7   glucose         94 non-null     float64
 8   disease_risk    100 non-null    int64
dtypes: float64(5), int64(1), object(3)
memory usage: 7.2+ KB
```

Statistical summary: -

memory usage: 7.2+ KB

```python
# Statistical Summary
df.describe()
```

	age	bmi	blood_pressure	cholesterol	glucose	disease_risk
count	94.000000	94.000000	100.000000	94.000000	94.000000	100.000000
mean	49.553191	24.648051	120.488072	197.366298	101.679514	0.520000
std	18.183424	5.178796	16.506529	31.224124	21.030685	0.502117
min	21.000000	10.602716	83.151103	99.077134	47.824240	0.000000
25%	34.000000	21.341209	109.829057	176.343199	88.119526	0.000000
50%	47.500000	24.976118	121.280470	195.392772	101.924590	1.000000
75%	66.000000	28.123443	132.044203	222.634167	117.333089	1.000000
max	79.000000	36.366726	167.805500	264.296156	143.456846	1.000000

Missing value summary

```
# Missing Values Summary
df.isnull().sum()
```

```
patient_id        0
age               6
gender            6
region            6
bmi               6
blood_pressure    0
cholesterol       6
glucose           6
disease_risk      0
dtype: int64
```

Missing value imputation: -

Method 1: Simple Imputer(mean)

```
# Missing Value Imputation
# Method 1: Simple Imputer (Mean)

from sklearn.impute import SimpleImputer

mean_imp = SimpleImputer(strategy="mean")

df["bmi"] = mean_imp.fit_transform(df[["bmi"]]).ravel()

df["bmi"].isnull().sum()
```

```
np.int64(0)
```

Method 2: Most Frequent Imputer

```
# Method 2: Most Frequent Imputer

cat_imp = SimpleImputer(
    strategy="most_frequent"
)

df["gender"] = cat_imp.fit_transform(
    df[["gender"]]
).ravel()

df["region"] = cat_imp.fit_transform(
    df[["region"]]
).ravel()

print(df[["gender","region"]].isnull().sum())
```

```
gender    0
region    0
dtype: int64
```

Method 3: Missing Indicator + Random Sample

```
# Method 3: Missing Indicator + Random Sample

df["age_missing"] = df["age"].isnull().astype(int)

sample = df["age"].dropna().sample(
    df["age"].isnull().sum(),
    random_state=42
)

df.loc[df["age"].isnull(),"age"] = sample.values

df.head()
```

	patient_id	age	gender	region	bmi	blood_pressure	cholesterol	glucose	disease_risk	age_missing
0	P1	74.0	Female	East	23.928378	111.660827	159.730615	99.967281	0	1
1	P2	40.0	Male	East	21.554029	118.083978	99.077134	91.479353	0	1
2	P3	69.0	Female	North	28.639166	96.591644	233.706253	125.509265	1	1
3	P4	55.0	Male	West	21.904653	150.005221	264.296156	98.168679	0	1
4	P5	40.0	Female	North	26.780799	101.998024	189.722334	118.741450	0	1

Method 4 : KNN Imputer

```python
# Method 4: KNN Imputer
from sklearn.impute import KNNImputer

knn = KNNImputer()

cols = [
    "age",
    "bmi",
    "blood_pressure",
    "cholesterol",
    "glucose"
]

df[cols] = knn.fit_transform(df[cols])
print(df)
```

```
   patient_id   age  gender region        bmi  blood_pressure  cholesterol  \
0          P1  74.0  Female   East  23.928378      111.660827   159.730615
1          P2  40.0    Male   East  21.554029      118.083978    99.077134
2          P3  69.0  Female  North  28.639166       96.591644   233.706253
3          P4  55.0    Male   West  21.904653      150.005221   264.296156
4          P5  40.0  Female  North  26.780799      101.998024   189.722334
..        ...   ...     ...    ...        ...             ...          ...
95        P96  33.0  Female   West  27.338585      115.525482   176.627593
96        P97  50.0    Male  South  17.242830      113.830641   173.836887
97        P98  67.0    Male   West  22.447534      134.760353   179.869283
98        P99  34.0    Male   East  21.335244      121.553521   210.078462
99       P100  27.0    Male   West  25.976732      124.014540   214.832279

       glucose  disease_risk  age_missing
0    99.967281             0            1
1    91.479353             0            1
2   125.509265             1            1
3    98.168679             0            1
4   118.741450             0            1
..         ...           ...          ...
95  121.349123             0            0
96  114.229085             0            0
97   94.212441             0            0
98   71.695950             0            0
99  111.908023             0            0

[100 rows x 10 columns]
```

Method 5: MICE

```python
# Method 5: MICE
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

mice = IterativeImputer(random_state=42)
df[cols] = mice.fit_transform(df[cols])
print(df)
```

```
   patient_id   age  gender region        bmi  blood_pressure  cholesterol  \
0          P1  74.0  Female   East  23.928378      111.660827   159.730615
1          P2  40.0    Male   East  21.554029      118.083978    99.077134
2          P3  69.0  Female  North  28.639166       96.591644   233.706253
3          P4  55.0    Male   West  21.904653      150.005221   264.296156
4          P5  40.0  Female  North  26.780799      101.998024   189.722334
..        ...   ...     ...    ...        ...             ...          ...
95        P96  33.0  Female   West  27.338585      115.525482   176.627593
96        P97  50.0    Male  South  17.242830      113.830641   173.836887
97        P98  67.0    Male   West  22.447534      134.760353   179.869283
98        P99  34.0    Male   East  21.335244      121.553521   210.078462
99       P100  27.0    Male   West  25.976732      124.014540   214.832279

       glucose  disease_risk  age_missing
0    99.967281             0            1
1    91.479353             0            1
2   125.509265             1            1
3    98.168679             0            1
4   118.741450             0            1
..         ...           ...          ...
95  121.349123             0            0
96  114.229085             0            0
97   94.212441             0            0
98   71.695950             0            0
99  111.908023             0            0

[100 rows x 10 columns]
```

Outlier detection:

z-score method:

```python
# Outlier Detection
# Z-Score Method
from scipy.stats import zscore

df["zscore_bmi"] = zscore(df["bmi"])

outliers = df[
    abs(df["zscore_bmi"]) > 3
]

print("Outliers:",len(outliers))
```

```
Outliers: 0
```

IQR method:

```
# IQR Method
Q1 = df["bmi"].quantile(0.25)
Q3 = df["bmi"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

print(lower, upper)
```

```
12.401990797759918 36.677475734546576
```

Percentile method:

```
# Percentile Method
print(df["bmi"].quantile(0.01))
print(df["bmi"].quantile(0.99))
```

```
12.631017773241775
35.60215400843934
```

Outlier treatment:

```
# Outlier Treatment
from scipy.stats.mstats import winsorize

df["bmi"] = winsorize(
    df["bmi"],
    limits=[0.01,0.01]
)

df.head()
```

	patient_id	age	gender	region	bmi	blood_pressure	cholesterol	glucose	disease_risk	age_missing	zscore_bmi
0	P1	74.0	Female	East	23.928378	111.660827	159.730615	99.967281	0	1	-0.144100
1	P2	40.0	Male	East	21.554029	118.083978	99.077134	91.479353	0	1	-0.619517
2	P3	69.0	Female	North	28.639166	96.591644	233.706253	125.509265	1	1	0.799142
3	P4	55.0	Male	West	21.904653	150.005221	264.296156	98.168679	0	1	-0.549311
4	P5	40.0	Female	North	26.780799	101.998024	189.722334	118.741450	0	1	0.427041

Final dataset validation:

```python
# Final Dataset Validation
df.isnull().sum()
```

```
patient_id        0
age               0
gender            0
region            0
bmi               0
blood_pressure    0
cholesterol       0
glucose           0
disease_risk      0
age_missing       0
zscore_bmi        0
dtype: int64
```

```python
df.head()
```

	patient_id	age	gender	region	bmi	blood_pressure	cholesterol	glucose	disease_risk	age_missing	zscore_bmi
0	P1	74.0	Female	East	23.928378	111.660827	159.730615	99.967281	0	1	-0.144100
1	P2	40.0	Male	East	21.554029	118.083978	99.077134	91.479353	0	1	-0.619517
2	P3	69.0	Female	North	28.639166	96.591644	233.706253	125.509265	1	1	0.799142
3	P4	55.0	Male	West	21.904653	150.005221	264.296156	98.168679	0	1	-0.549311
4	P5	40.0	Female	North	26.780799	101.998024	189.722334	118.741450	0	1	0.427041